

Compiler Design

Lab Assessment 4

Student Name: Avyakth Shriram

Register Number: 24BCE5461

PROGRAM 1

Semantic Analyser for Type Checking

Aim:

To design and implement a semantic analyser using Lex and YACC that checks for type consistency in variable declarations, expressions, and assignment statements, reporting semantic errors such as undeclared variables, variable redeclarations, and type mismatches.

Procedure:

- 1) **Define Tokens in Lex (exp7.l):** Recognize type keywords (int, float, char), identifiers, integer constants, floating-point constants, character literals, and arithmetic/assignment operators.
- 2) **Design the Symbol Table in YACC (exp7.y):** Implement an internal symbol table structure storing variable identifiers and their corresponding data types.
- 3) **Declare Context and Precedence:** Declare operator precedence (*, / over +, -) and assign semantic types (<type>, <id>) to grammar symbols.
- 4) **Enforce Semantic Rules:**
 - a) **Declaration Rule:** When a variable is declared, verify whether it already exists in the symbol table. If so, raise a Redclaration error; otherwise, store its name and type.
 - b) **Expression Evaluation Rule:** Recursively check operand types. Operands of differing types trigger a Type Mismatch error. Valid operations propagate the resolved type.

c) Assignment Rule: Lookup the LHS identifier. If not declared, throw an Undeclared Variable error. If declared, compare LHS type against RHS expression type; report a Type Mismatch error if they disagree.

5) Compile and Run: Compile with Lex and YACC, then execute interactive test cases.

Lex/Yacc Program Code:

exp7.l

```
%{
#include "y.tab.h"
#include <string.h>
%}

%%

"int"          { return INT; }
"float"        { return FLOAT; }
"char"         { return CHAR; }

[0-9]+\.[0-9]+  { return FLOAT_NUM; }
[0-9]+          { return INT_NUM; }
'[a-zA-Z0-9]'\  { return CHAR_CONST; }

[a-zA-Z_][a-zA-Z0-9_]* {
    strncpy(yyval.id, yytext, 31);
    yyval.id[31] = '\0';
    return ID;
}

[ \t\r\n]+     { /* ignore whitespace */ }
.              { return yytext[0]; }
%%

int yywrap() {
    return 1;
}
```

exp7.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define T_INT 1
#define T_FLOAT 2
#define T_CHAR 3
#define T_ERROR -1

struct Symbol {
    char name[32];
    int type;
} symTab[100];

int symCount = 0;
int current_decl_type = 0;

int lookup(const char *name) {
    for (int i = 0; i < symCount; i++) {
        if (strcmp(symTab[i].name, name) == 0)
            return symTab[i].type;
    }
    return 0; // 0 indicates undeclared
}

int insert(const char *name, int type) {
    if (lookup(name) != 0) return 0; // Redclaration
    strcpy(symTab[symCount].name, name);
    symTab[symCount].type = type;
    symCount++;
    return 1;
}

const char* getTypeName(int type) {
    switch(type) {
        case T_INT: return "int";
        case T_FLOAT: return "float";
        case T_CHAR: return "char";
        default: return "error/unknown";
    }
}
```

```

    }
}

int yylex();
void yyerror(const char *s);
%}

%union {
    int type;
    char id[32];
}

%token <type> INT FLOAT CHAR
%token <id> ID
%token INT_NUM FLOAT_NUM CHAR_CONST

%type <type> type expr

%left '+' '-'
%left '*' '/'

%%

program:
    program statement
    | /* empty */
    ;

statement:
    declaration
    | assignment
    | error ';' { yyerrok; }
    ;

declaration:
    type var_list ';'
    ;

type:
    INT { current_decl_type = T_INT; $$ = T_INT; }
    | FLOAT { current_decl_type = T_FLOAT; $$ = T_FLOAT; }
    | CHAR { current_decl_type = T_CHAR; $$ = T_CHAR; }
    ;

```

```

var_list:
    var_list ',' ID {
        if (!insert($3, current_decl_type)) {
            printf(">> [SEMANTIC ERROR] Redeclaration of identifier '%s'\n", $3);
        } else {
            printf(">> [SUCCESS] Declared '%s' as %s\n", $3,
getTypeName(current_decl_type));
        }
    }
    | ID {
        if (!insert($1, current_decl_type)) {
            printf(">> [SEMANTIC ERROR] Redeclaration of identifier '%s'\n", $1);
        } else {
            printf(">> [SUCCESS] Declared '%s' as %s\n", $1,
getTypeName(current_decl_type));
        }
    }
;

```

assignment:

```

ID '=' expr ';' {
    int idType = lookup($1);
    if (idType == 0) {
        printf(">> [SEMANTIC ERROR] Variable '%s' used without prior
declaration\n", $1);
    } else if ($3 == T_ERROR) {
        printf(">> [SEMANTIC ERROR] Assignment aborted due to expression type
errors\n");
    } else if (idType != $3) {
        printf(">> [SEMANTIC ERROR] Type mismatch: Cannot assign '%s'
expression to '%s' variable '%s'\n",
getTypeName($3), getTypeName(idType), $1);
    } else {
        printf(">> [SUCCESS] Valid assignment: '%s' = (%s value)\n", $1,
getTypeName(idType));
    }
}
;

```

expr:

```

expr '+' expr {
    if ($1 == T_ERROR || $3 == T_ERROR) $$ = T_ERROR;
    else if ($1 != $3) {

```

```

        printf(">> [SEMANTIC ERROR] Type mismatch on '+': '%s' and '%s'\n",
               getTypeName($1), getTypeName($3));
        $$ = T_ERROR;
    } else $$ = $1;
}
| expr '-' expr {
    if ($1 == T_ERROR || $3 == T_ERROR) $$ = T_ERROR;
    else if ($1 != $3) {
        printf(">> [SEMANTIC ERROR] Type mismatch on '-': '%s' and '%s'\n",
               getTypeName($1), getTypeName($3));
        $$ = T_ERROR;
    } else $$ = $1;
}
| expr '*' expr {
    if ($1 == T_ERROR || $3 == T_ERROR) $$ = T_ERROR;
    else if ($1 != $3) {
        printf(">> [SEMANTIC ERROR] Type mismatch on '*': '%s' and '%s'\n",
               getTypeName($1), getTypeName($3));
        $$ = T_ERROR;
    } else $$ = $1;
}
| expr '/' expr {
    if ($1 == T_ERROR || $3 == T_ERROR) $$ = T_ERROR;
    else if ($1 != $3) {
        printf(">> [SEMANTIC ERROR] Type mismatch on '/': '%s' and '%s'\n",
               getTypeName($1), getTypeName($3));
        $$ = T_ERROR;
    } else $$ = $1;
}
| '(' expr ')' { $$ = $2; }
| ID {
    int t = lookup($1);
    if (t == 0) {
        printf(">> [SEMANTIC ERROR] Identifier '%s' is undeclared\n", $1);
        $$ = T_ERROR;
    } else $$ = t;
}
| INT_NUM { $$ = T_INT; }
| FLOAT_NUM { $$ = T_FLOAT; }
| CHAR_CONST { $$ = T_CHAR; }
;
%%

```

```

void yyerror(const char *s) {
    printf(">> [SYNTAX ERROR] Invalid statement syntax\n");
}

int main() {

printf("=====
=====\\n");
    printf("    EXP 7: SEMANTIC ANALYSER TYPE CHECKER (LEX & YACC)
\\n");
    printf(" Enter declarations and expressions ending with ';':          \\n");
    printf(" Example:                                \\n");
    printf("  int a, b;                                \\n");
    printf("  float x;                                \\n");
    printf("  a = b + 10;                                \\n");
    printf("  a = x;                                \\n");
    printf(" Press Ctrl+C or Ctrl+D to terminate          \\n");

printf("=====
=====\\n\\n");
    yyparse();
    return 0;
}

```

Screenshots & Output:

```
avyakths@AvysLegionSlim5:~  
11:30:25 avyakths@AvysLegionSlim5 Compiler-Design-Lab #|main ↵ ls  
Lab_Assignment_1 Lab_Assignment_2 Lab_Assignment_3 Lab_Assignment_4  
11:31:34 avyakths@AvysLegionSlim5 Compiler-Design-Lab #|main ↵ cd Lab_Assignment_4  
/home/avyakths/Compiler-Design-Lab/Lab_Assignment_4  
11:31:50 avyakths@AvysLegionSlim5 Lab_Assignment_4 #|main ↵ ls  
Experiment_10 Experiment_7 Experiment_8 Experiment_9  
11:31:53 avyakths@AvysLegionSlim5 Lab_Assignment_4 #|main ↵ cd Experiment_7  
/home/avyakths/Compiler-Design-Lab/Lab_Assignment_4/Experiment_7  
11:32:16 avyakths@AvysLegionSlim5 Experiment_7 #|main ↵ ls  
exp7 exp7.l exp7.y lex.yy.c y.tab.c y.tab.h  
11:32:17 avyakths@AvysLegionSlim5 Experiment_7 #|main ↵ ./exp7  
=====
```

EXP 7: SEMANTIC ANALYSER TYPE CHECKER (LEX & YACC)
Enter declarations and expressions ending with ';' :
Example:
int a, b;
float x;
a = b + 10;
a = x;
Press Ctrl+C or Ctrl+D to terminate
=====

```
int a, b;  
>> [SUCCESS] Declared 'a' as int  
>> [SUCCESS] Declared 'b' as int  
float x, y;  
>> [SUCCESS] Declared 'x' as float  
>> [SUCCESS] Declared 'y' as float  
char ch;  
>> [SUCCESS] Declared 'ch' as char  
a = 10;  
>> [SUCCESS] Valid assignment: 'a' = (int value)  
x = 3.14;  
>> [SUCCESS] Valid assignment: 'x' = (float value)  
ch = 'z';  
>> [SUCCESS] Valid assignment: 'ch' = (char value)  
a = a + b * 2;  
>> [SUCCESS] Valid assignment: 'a' = (int value)  
int a;  
>> [SEMANTIC ERROR] Redeclaration of identifier 'a'  
int a;  
>> [SEMANTIC ERROR] Redeclaration of identifier 'a'  
k = 10;  
>> [SEMANTIC ERROR] Variable 'k' used without prior declaration  
int p;  
>> [SUCCESS] Declared 'p' as int  
float q;  
>> [SUCCESS] Declared 'q' as float  
p = q;  
>> [SEMANTIC ERROR] Type mismatch: Cannot assign 'float' expression to 'int' variable 'p'  
int num;  
>> [SUCCESS] Declared 'num' as int  
char letter;  
>> [SUCCESS] Declared 'letter' as char  
num = num + letter;  
>> [SEMANTIC ERROR] Type mismatch on '+': 'int' and 'char'  
>> [SEMANTIC ERROR] Assignment aborted due to expression type errors  
^C
```

PROGRAM 2

Checking Balanced Brackets and Parentheses

Aim:

To design and implement a parser using Lex and YACC to verify whether parentheses (), square brackets [], and curly braces {} are present in balanced numbers and properly nested within a given statement.

Procedure:

- 1) **Define Tokens in Lex (exp8.l):** Recognize opening and closing brackets ((,), [,], {, }) and newline characters (\n). Filter out and ignore variables, identifiers, operators, and whitespace.
- 2) **Formulate the Dyck Grammar in YACC (exp8.y):**
 - a) Specify the context-free grammar:

$$S \rightarrow (S)S \mid [S]S \mid \{S\}S \mid \epsilon$$

- b) This grammar captures all balanced permutations of mixed brackets without ambiguity or shift/reduce conflicts.
- 3) **Parse Interactive Lines:** Read inputs line-by-line using a rule $line \rightarrow S \text{ 'n'}$.
- 4) **Detect Errors:** Use YACC's error '\n' recovery mechanism. An unmatched bracket, missing closing bracket, or illegal cross-nesting (e.g., ([)]) halts the reduction, triggering the error rule and resetting parsing state via `yyerrok`.
- 5) **Output Verdict:** Display whether each evaluated line is VALID or INVALID.

Lex/Yacc Program Code:

exp8.l

```
%{
#include "y.tab.h"
}%

%%

"("  { return '('; }
")"  { return ')'; }
"["  { return '['; }
"]"  { return ']'; }
"{"  { return '{'; }
"}"  { return '}'; }
"\n" { return '\n'; }
[ \t]+ { /* ignore spaces and tabs */ }
.      { /* ignore variables, identifiers, numbers, operators */ }
%%

int yywrap() {
    return 1;
}
```

exp8.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
}%

%%

input:
    input line
    | /* empty */
    ;

line:
```

```

expr '\n' {
    printf(">> RESULT: VALID (All brackets & parentheses are balanced)\n\n");
}
| '\n' {
    /* silently ignore empty lines */
}
| error '\n' {
    printf(">> RESULT: INVALID (Unbalanced or improperly nested
brackets)\n\n");
    yyerrok;
}
;

```

```

expr:
    '(' S ')' S
    | '[' S ']' S
    | '{' S '}' S
;

```

```

S:
    '(' S ')' S
    | '[' S ']' S
    | '{' S '}' S
    | /* empty */
;
%%

```

```

void yyerror(const char *s) {
    /* Errors are captured and reported via the error '\n' rule */
}

```

```

int main() {

```

```

    printf("=====
=====\\n");

```

```

    printf("  EXP 8: BALANCED BRACKETS & PARENTHESES CHECKER
(LEX/YACC)  \\n");
    printf("  Supports: ( ), [ ], { } inside expressions or code lines  \\n");
    printf("  Press Ctrl+C or Ctrl+D to exit                      \\n");

```

```

    printf("=====
=====\\n\\n");

```

```

    printf("Enter statements/bracket sequences:\\n");

```

```
yyparse();  
return 0;  
}
```

Screenshots & Output:

```
avyakths@AvysLegionSlim5:~  
11:59:26 avyakths@AvysLegionSlim5 Lab_Assignment_4 #|main ↵ ls  
Experiment_10 Experiment_7 Experiment_8 Experiment_9  
11:59:28 avyakths@AvysLegionSlim5 Lab_Assignment_4 #|main ↵ cd Experiment_8  
/home/avyakths/Compiler-Design-Lab/Lab_Assignment_4/Experiment_8  
11:59:46 avyakths@AvysLegionSlim5 Experiment_8 #|main ↵ ls  
exp8 exp8.l exp8.y lex.yy.c y.tab.c y.tab.h  
11:59:48 avyakths@AvysLegionSlim5 Experiment_8 #|main ↵ ./exp8  
=====
```

EXP 8: BALANCED BRACKETS & PARENTHESES CHECKER (LEX/YACC)
Supports: (), [], { } inside expressions or code lines
Press Ctrl+C or Ctrl+D to exit

```
=====
```

Enter statements/bracket sequences:
(a + b) * [c - d]
>> RESULT: VALID (All brackets & parentheses are balanced)

{ int arr[10]; (x + y); }
>> RESULT: VALID (All brackets & parentheses are balanced)

{{(())}}

>> RESULT: VALID (All brackets & parentheses are balanced)

(){}(){}()

>> RESULT: VALID (All brackets & parentheses are balanced)

(a + b

>> RESULT: INVALID (Unbalanced or improperly nested brackets)

([]

>> RESULT: INVALID (Unbalanced or improperly nested brackets)

{ [}]

>> RESULT: INVALID (Unbalanced or improperly nested brackets)

) (

>> RESULT: INVALID (Unbalanced or improperly nested brackets)

((a + b)

>> RESULT: INVALID (Unbalanced or improperly nested brackets)

^C
12:34:01 avyakths@AvysLegionSlim5 Experiment_8 #|main ↵ |

THANK YOU